# Proyecto Integrador Plataforma Interoperable de Trámites Migratorios

# INGENIERÍA DE SOFTWARE III

## Documento Acumulativo del Proyecto Integrador

Curso: Ingeniería de Software III

**Docente:** María Félix Mosquera

**Modalidad:** Scrum con entregas semanales

Corte de este documento: Sprint 2 - Entorno, modelo de datos y autenticación (TeamTrack, sprint 2, 28/09/2026 - 04/10/2026)

**Duración:** 10 – 13 semanas

## 🧩 1. Información del Equipo

Caso seleccionado: Trámites Migratorios Lentos | Subasta: 450,000 Neuronix | Saldo: 550,000 Neuronix

Saldo operativo al cierre del Sprint 2: 290.672 Neuronix disponibles de los 350.000 Neuronix del presupuesto operativo (la subasta de 450.000 Neuronix y el fondo de herramientas de 200.000 Neuronix se mantienen separados)

- Kenneth Carter - Rol Scrum: Scrum Master y DevOps
- Richard Rodríguez - Rol Scrum: Product Owner
- Edgar Rosario - Rol Scrum: Desarrollador Backend
- Jonathan Gómez - Rol Scrum: Desarrollador Frontend y UX
- Vivian Paris - Rol Scrum: QA y Seguridad

Distribución del esfuerzo del Sprint 2 (TeamTrack): Valoración por puntos: Edgar Rosario 48 h - Vivian Paris 8 h - Kenneth Carter 5 h - Jonathan Gómez 5 h - Richard Rodríguez 2 h - Total 68 h. TeamTrack conserva 34,5 h de ejecución real.

## 📌 2. Sprint 0 - Inicio del Proyecto

### 2.1. Alcance del Proyecto

Sistema web para registrar, gestionar y dar seguimiento a trámites migratorios mediante expedientes digitales y acceso por roles.

Descripción: La Plataforma Interoperable de Trámites Migratorios busca reducir demoras causadas por documentos repetidos y validaciones manuales. Centralizará solicitudes, expedientes y documentos, y permitirá centralizar expedientes y registrar verificaciones documentales con datos ficticios. Las instituciones se representan como fuentes externas; el MVP no realiza conexiones reales ni transferencias automáticas. Se trabajará solo con datos ficticios para las demostraciones académicas.

### 2.2. Historias de Usuario

Complete la tabla:

| ID | Historia de Usuario | Criterios de Aceptación |
|---|---|---|
| HU-01 | Como solicitante, quiero registrarme e iniciar sesión para entrar a mi cuenta. | Credenciales válidas permiten entrar; datos vacíos o incorrectos muestran un error. |
| HU-02 | Como solicitante, quiero crear un expediente y adjuntar documentos para iniciar mi trámite. | El sistema genera un expediente y permite asociar documentos de prueba. |
| HU-03 | Como funcionario, quiero revisar y actualizar el estado de los expedientes para dar seguimiento. | El funcionario ve los expedientes autorizados y registra un nuevo estado. |

### 2.3. Product Backlog Inicial

| ID | Descripción | Prioridad | Estimación |
|---|---|---|---|
| PB-01 | Registro e inicio de sesión con roles | Alta | 8 puntos |
| PB-02 | Crear expediente y cargar documentos | Alta | 13 puntos |
| PB-03 | Panel para consultar estado del trámite | Alta | 8 puntos |
| PB-04 | Panel de funcionario para revisar expedientes | Media | 13 puntos |
| PB-05 | Validación de documentos y estados | Alta | 13 puntos |
| PB-06 | Auditoría de acciones y notificaciones | Media | 8 puntos |
| PB-07 | Reportes e indicadores administrativos | Media | 8 puntos |
| PB-08 | Pruebas, seguridad y despliegue final | Alta | 13 puntos |

### 2.4. Pregunta Cuestionadora del Cliente

¿Cómo diseñar un sistema interoperable que reduzca los tiempos de los trámites migratorios sin comprometer la seguridad de los datos personales?

Primera hipótesis de solución: una plataforma web con APIs REST, expedientes digitales, control de acceso por roles y una capa de adaptadores. En el MVP la interoperabilidad se demuestra mediante verificaciones manuales asistidas y servicios simulados, manteniendo trazabilidad de las acciones.

Visión técnica inicial: frontend web, backend con Java y Spring Boot, base de datos PostgreSQL, autenticación por roles y ejecución integrada mediante Docker.

### 2.5. Mapa Inicial de Riesgos

Riesgo 1 - Protección de datos personales: se mitigará usando cuentas de prueba, acceso por roles y evitando datos reales.

Riesgo 2 - Integración entre instituciones: para el proyecto académico se utilizarán APIs y datos ficticios que representen el intercambio seguro.

Riesgo 3 - Retrasos por integración tardía: se harán pruebas de frontend, backend y base de datos desde los primeros sprints.

Riesgo 4 - Coordinación del equipo: se usarán tareas asignadas, reuniones cortas y seguimiento en TeamTrack.

### 2.6. Daily Scrum del Sprint 0

#### ¿Qué hicimos desde el último laboratorio?

Analizamos el caso adquirido en la subasta, definimos los roles del equipo, construimos las historias de usuario y el backlog inicial, y elaboramos la presentación y el video del Sprint 0.

#### ¿Qué haremos para el próximo laboratorio?

Desarrollar el acceso por roles, las pantallas iniciales, la API y la base de datos de prueba. Cada integrante avanzará en su rol y se entregarán evidencias en el repositorio, TeamTrack y el documento acumulativo.

#### ¿Qué problemas o bloqueos tuvimos?

Al inicio fue necesario aclarar el alcance del caso, cómo representar la interoperabilidad entre instituciones y cómo proteger datos sensibles. También identificamos la necesidad de mejorar la coordinación de fechas y avances.

Métricas iniciales: 5 integrantes, 3 historias de usuario iniciales, 8 elementos de backlog, 450,000 Neuronix invertidos en la subasta y 550,000 Neuronix disponibles.

Este es el backlog de arranque; el repositorio oficial lo refinó después a 10 épicas, 36 historias, 7 sprints y 229 puntos.

### 2.7. Factura de Consultoría del Sprint 0

Sprint facturado: Sprint 0 - Análisis de problema.

Actividades realizadas: análisis del caso, definición de roles, alcance, historias de usuario, backlog inicial, arquitectura inicial, presentación y video.

Evidencias entregadas: documento acumulativo, presentación en PDF, video de exposición y material de análisis.

Horas estimadas: 15 horas de trabajo grupal estimado.

Roles involucrados: Product Owner, Scrum Master y DevOps, Backend, Frontend y UX, QA y Seguridad.

Métricas aplicadas: historias definidas, elementos priorizados del backlog, integrantes participantes y presupuesto Neuronix.

Valor solicitado en Neuronix: Pendiente de aprobación del equipo y validación de la docente.

## 3. Sprint 1 - Avance de Desarrollo y Correcciones

Este Sprint documenta el paso del análisis inicial a un avance técnico verificable. El equipo organizó sus responsabilidades, integró los componentes principales de la aplicación y comprobó un recorrido de prueba con datos ficticios. Las actividades y valores indicados en esta sección están sujetos a la revisión de evidencias del laboratorio.

### 3.1. Objetivo del Sprint

Construir una base funcional de la Plataforma Interoperable de Trámites Migratorios, aplicando las correcciones del Sprint 0 e integrando frontend, backend, base de datos y entorno Docker para demostrar un flujo básico por roles.

### 3.2. Correcciones Aplicadas del Sprint 0

| Aspecto | Situación inicial | Corrección aplicada y evidencia |
|---|---|---|
| Backlog | El trabajo estaba planteado de forma general. | Se organizó el Sprint 1 en TeamTrack con tareas, responsables y seguimiento. |
| Roles | Las funciones estaban definidas, pero sin evidencia ni métricas asociadas. | Cada integrante cuenta con responsabilidad, herramienta, evidencia esperada y métrica inicial. |
| Visión técnica | La solución se encontraba en una propuesta conceptual. | Se integró un entorno local con frontend, Java y Spring Boot, PostgreSQL y Docker. |
| Riesgos | La protección de datos y la integración estaban identificadas de forma inicial. | Se usan datos ficticios, control por roles, revisión de integración y simulación de servicios externos. |

### 3.3. Daily Scrum del Sprint 1

#### ¿Qué hicimos desde el último laboratorio?

Se corrigió el backlog y se consolidó la organización del trabajo mediante repositorio, ramas y TeamTrack.

Se construyó la base de pantallas por rol y las primeras APIs para el proceso de trámites.

Se integraron frontend, Java con Spring Boot, PostgreSQL y Docker en un entorno local.

Se comprobó un recorrido de demostración: solicitante crea expediente y asocia documento; funcionario revisa y actualiza el estado; solicitante consulta el cambio; administrador consulta el panel.

#### ¿Qué haremos para el próximo laboratorio?

Completar la autenticación y la redirección del inicio de sesión según el rol.

Documentar casos de prueba funcionales y de seguridad, incluyendo un registro inicial de defectos.

Registrar métricas de requisitos cubiertos, pruebas ejecutadas, defectos y horas por tarea.

Preparar una línea base de pruebas con Apache JMeter y revisión de seguridad con OWASP ZAP.

**¿Qué problemas o bloqueos tuvimos?**

La integración de ramas exigió revisar que frontend, backend y entorno Docker utilizaran el mismo flujo.

No se utilizarán datos reales ni conexiones a instituciones reales; la interoperabilidad se representa mediante servicios y datos ficticios.

La ejecución de pruebas de carga y seguridad queda como actividad del siguiente control, ya que este Sprint priorizó el recorrido funcional integrado.

### 3.4. Roles Formales del Equipo

| Integrante y rol | Responsabilidad y artefacto | Herramienta y métrica |
|---|---|---|
| Richard Rodríguez<br>Product Owner | Prioriza el backlog y valida criterios de aceptación. Evidencia: historias y tablero actualizado. | TeamTrack y GitHub.<br>Métrica: requisitos priorizados. |
| Edgar Rosario<br>Dev Lead y Backend | Implementa APIs, lógica de negocio y persistencia. Evidencia: servicios y conexión a base de datos. | Java, Spring Boot y PostgreSQL.<br>Métrica: endpoints probados. |
| Jonathan Gómez<br>Frontend y UX | Implementa pantallas y navegación por rol. Evidencia: prototipo navegable y flujo de inicio de sesión. | HTML, CSS y JavaScript.<br>Métrica: pantallas funcionales. |
| Vivian Paris<br>QA y Seguridad | Define casos de prueba y revisa riesgos de seguridad. Evidencia: checklist y resultados. | Apache JMeter y OWASP ZAP.<br>Métrica: casos y defectos registrados. |
| Kenneth Carter<br>Scrum Master y DevOps | Coordina el Sprint e integra un entorno reproducible. Evidencia: entorno Docker y seguimiento. | Docker y GitHub.<br>Métrica: integración y tareas cerradas. |

### 3.5. Incremento Técnico Entregado

La aplicación se ejecuta como una plataforma web de demostración. El frontend está construido con HTML, CSS y JavaScript; el backend utiliza Java con Spring Boot; la información de prueba se persiste en PostgreSQL; y Docker permite levantar el entorno de forma integrada. El flujo validado utiliza los roles Solicitante, Funcionario y Administrador.

Solicitante: inicia sesión, crea un expediente y asocia un documento de prueba.

Funcionario: revisa el expediente y cambia el estado del trámite.

Solicitante: consulta el estado actualizado de su trámite.

Administrador: consulta el panel de usuarios para la demostración.

### 3.6. Evidencias de Calidad, Seguridad y DevOps

| Área | Evidencia del Sprint 1 | Siguiente actividad |
|---|---|---|
| Calidad | Comprobación manual del recorrido completo por roles con datos ficticios. | Documentar casos de prueba y resultados. |
| Seguridad | Uso de datos ficticios y control de acceso por roles como decisión de diseño. | Preparar revisión con OWASP ZAP. |
| DevOps | Entorno local integrado mediante Docker y revisión de cambios antes de integrar. | Formalizar automatización CI CD cuando el flujo sea estable. |
| Rendimiento | Herramienta Apache JMeter adquirida para pruebas de carga. | Definir escenario de usuarios y línea base. |

### 3.7. Métricas Iniciales y Riesgos Actualizados

| Indicador | Valor o estado | Uso en el seguimiento |
|---|---|---|
| Equipo formalizado | 5 integrantes con rol asignado. | Verificar participación y responsabilidades. |
| Flujo de demostración | 3 roles involucrados: solicitante, funcionario y administrador. | Confirmar trazabilidad del trámite. |
| Componentes integrados | 4 componentes: frontend, backend, PostgreSQL y Docker. | Comprobar que el entorno sea reproducible. |
| Tareas del Sprint 1 | Registradas y seguidas en TeamTrack. | Revisar avance y bloqueos por responsable. |
| Saldo operativo actual | 280,784 Neuronix disponibles después de facturar los Sprints 1 y 2; la subasta de 450,000 y la inversión de 200,000 en herramientas se mantienen separadas. | Controlar presupuesto para los siguientes sprints. |

Riesgo de integración: se mitiga mediante revisión de cambios e integración progresiva desde develop.

Riesgo de datos personales: se mitiga utilizando únicamente cuentas, expedientes y documentos ficticios.

Riesgo de interoperabilidad externa: se mitiga con servicios simulados y verificaciones manuales asistidas; las conexiones reales quedan sujetas a convenios y autorizaciones.

### 3.8. Retrospectiva del Sprint 1

El equipo logró convertir la propuesta técnica inicial en un entorno demostrable y con roles definidos. Para el siguiente Sprint se debe fortalecer la autenticación por rol, formalizar los casos de prueba y comenzar a registrar métricas de esfuerzo y defectos. También se debe mantener la coordinación de integración para evitar diferencias entre las ramas.

### 3.9. Factura de Consultoría del Sprint 1

La siguiente factura registra el valor propuesto por el trabajo realizado. Su aprobación depende de las evidencias del Sprint, la participación del equipo y la validación realizada en el laboratorio.

| Concepto | Detalle |
|---|---|
| Proyecto y Sprint | Plataforma Interoperable de Trámites Migratorios. Sprint 1 - Avance de Desarrollo y Correcciones. |
| Cliente | Neuronix. |
| Actividades realizadas | Correcciones del Sprint 0; organización del backlog; roles formales; prototipo; integración de frontend, backend, PostgreSQL y Docker; comprobación del flujo por roles. |
| Evidencias entregadas | Documento acumulativo actualizado; presentación Sprint 1; repositorio; TeamTrack; demostración local; flujo con datos ficticios. |
| Roles involucrados | Product Owner, Dev Lead y Backend, Frontend y UX, QA y Seguridad, Scrum Master y DevOps. |
| Horas estimadas | 38 horas de trabajo grupal estimado: Richard 6, Edgar 9, Jonathan 9, Vivian 6 y Kenneth 8. |
| Métricas aplicadas | 5 roles formales; 3 roles en el flujo; 4 componentes integrados; tareas registradas; inicio de control de pruebas y defectos. |
| Herramientas e inversión | Docker, Apache JMeter y OWASP ZAP. Inversión reportada: 200,000 Neuronix. |
| Valor solicitado del Sprint | 15,656 Neuronix. Estado: pendiente de aprobación según evidencias y participación. |
| Distribución propuesta | Reserva de empresa: 30% = 4,697 Neuronix. Fondo operativo del equipo: 70% = 10,959 Neuronix. Esta distribución se debe confirmar |

| | |
|---|---|
| | según la regla explicada en clase. |

## 4. Sprint 2 - Entorno, Modelo de Datos y Autenticación

El Sprint 2 es el punto de viraje del proyecto: el sistema deja de ser un prototipo con login simulado y pasa a ser un software con autenticación real, autorización por rol, base de datos persistente y un entorno reproducible. Todo el trabajo se registró en TeamTrack: 31 ítems cerrados (5 historias de usuario y 26 tareas), 34 story points y 34,5 horas de esfuerzo real grupal.

### 4.1. Objetivo del Sprint

Dotar al proyecto de un entorno reproducible y persistente, definir el modelo de datos del dominio migratorio y sustituir la autenticación simulada del Sprint 1 por autenticación y autorización reales, cubiertas por pruebas automatizadas y evidencia reproducible de cada decisión.

Período: 28 de septiembre al 4 de octubre de 2026. Alcance: historias de usuario HU-10 a HU-14 y 26 tareas de soporte, 34 story points y 34,5 horas de esfuerzo real grupal. La corrección central del Sprint es que el login simulado se sustituye por autenticación real con hashing, tabla de usuarios y control de acceso por rol; en paralelo se levanta el modelo de datos de 20 tablas con 4 migraciones versionadas y se automatiza la verificación con 68 pruebas en 11 clases.

### 4.2. Correcciones Aplicadas del Sprint 1

| Aspecto | Situación inicial | Corrección aplicada y evidencia |
|---|---|---|
| Autenticación | El login validaba las credenciales en el cliente y no existía sesión real. | Autenticación real con hashing BCrypt, token de sesión y tabla de usuarios. Verificado contra la API en ejecución: 401 NO_AUTENTICADO y 401 CREDENCIALES_INVALIDAS |

| | | |
|---|---|---|
| | | con mensaje genérico. |
| Autorización | El acceso a las áreas internas se controlaba únicamente en la interfaz. | Matriz de autorización por rol y endpoint publicada en Backend/docs/matriz-autorizacion.md. Verificado: AUDITOR es el único rol con acceso a GET /api/auditoria/eventos (los demás, 403) y ADMIN recibe 403 en GET /api/expedientes. |
| Persistencia | Los datos se conservaban en memoria del lado del cliente. | PostgreSQL con 20 tablas, 4 migraciones versionadas con Flyway y 9 usuarios semilla distribuidos en 5 roles. La API opera contra el esquema real. |
| Reproducibilidad | El arranque dependía de un entorno local no documentado. | Docker Compose con tres servicios (base de datos, backend e interfaz), health checks activos y orden de arranque documentado en scripts/README.md. |
| Calidad | La verificación era manual y dispersa, sin evidencia de ejecución continua. | 68 pruebas automatizadas en 11 clases ejecutadas con ./mvnw test: 0 fallos y 0 errores en la última corrida. |
| Manejo de errores | Los mensajes de error variaban según el endpoint y se confundían entre fallo de sesión y dato inválido. | Catálogo de errores estandarizado y publicado en Backend/docs/estandar-errores.md, con 400 ENTRADA_INVALIDA para datos inválidos y 401 differentiated para sesión. |
| Trazabilidad | El backlog del Sprint 1 se | Los 31 ítems del Sprint 2 |

| | | |
|---|---|---|
| | mantenía en un documento estático. | quedaron registrados en TeamTrack con responsable, esfuerzo en story points, horas asignadas y vínculo con el código. |

### 4.3. Daily Scrum del Sprint 2

**¿Qué hicimos desde el último laboratorio?**

- Se modeló y versionó el esquema de datos del dominio migratorio: 20 tablas, 4 migraciones Flyway y 9 usuarios semilla en 5 roles, sobre PostgreSQL en contenedor.
- Se implementó la autenticación real: hashing BCrypt de contraseñas, emisión y validación de token de sesión, y respuestas 401 diferenciadas para credenciales inválidas y sesión ausente.
- Se construyó la matriz de autorización por rol y se verificó endpoint por endpoint contra la API en ejecución, con evidencia de códigos 200, 401 y 403.
- Se levantó el sistema completo con Docker Compose: base de datos, backend e interfaz, con health checks y mapeo de puertos documentado.
- Se reportó la verificación con 68 pruebas unitarias y de integración en 11 clases, todas en verde en la última ejecución. Nota de trazabilidad: esta cifra corresponde a la última ejecución declarada en el corte del Sprint 2. Las mediciones posteriores deben identificarse con rama, fecha, commit y comando para que puedan reproducirse.
- Se consolidó la evidencia técnica del Sprint en el repositorio, con la especificación OpenAPI de 29 rutas y 37 operaciones, la matriz de autorización y el estándar de errores.
- Se corrigió la estimación del Sprint 1: el backlog oficial registra 50 puntos, por lo que su valoración coherente es 100 horas y 41,200 Neuronix. El Sprint 2 conserva 34 puntos y 28,016 Neuronix.

**¿Qué haremos para el próximo laboratorio?**

- Ejecutar el análisis de calidad estática con SonarQube sobre backend e interfaz, y publicar cobertura, deuda técnica y code smells como métricas del proyecto.
- Incorporar OWASP ZAP al ciclo de calidad para levantar y reportar vulnerabilidades de las superficies expuestas.
- Diseñar el plan de pruebas de carga con Apache JMeter sobre los endpoints de consulta de uso frecuente.
- Versionar en el repositorio los scripts de automatización de seguridad, el plan de pruebas de carga y la definición del pipeline de integración continua.
- Actualizar este documento acumulativo con las métricas de calidad estática y seguridad ya verificadas.

**¿Qué problemas o bloqueos tuvimos?**

- El arranque de la API requería definir variables de entorno (secreto de firma, credenciales de base de datos) por cada ambiente; se resolvió con un archivo de ejemplo y valores por defecto exclusivos de desarrollo local.
- La matriz de autorización rechazaba consultas legítimas del rol administrador hasta separar los casos de negocio de los casos de auditoría; se ajustó la tabla de permisos y se repitió la verificación completa.
- La primera versión del esquema mezclaba información de auditoría con la de los expedientes; se corrigió con una migración adicional que separa ambas responsabilidades.
- El contenedor de la interfaz no publicaba el puerto de forma estable en el 8081; se corrigió el mapeo de Docker Compose y se documentó en scripts/README.md.
- Las pruebas de integración dependían del orden de ejecución; se aislaron con datos de prueba propios para eliminar dependencias de estado entre clases.

### 4.4. Roles Formales del Equipo

| Integrante y rol | Responsabilidad y artefacto | Herramienta y métrica |
|---|---|---|
| Kenneth Carter<br>Scrum Master y DevOps Lead | Orquestación del Sprint 2, coordinación del Daily, cierre de los 31 ítems en TeamTrack y | TeamTrack, Docker Compose, GitHub Actions. Métrica: 31 ítems cerrados, 34 story points |

| | | |
|---|---|---|
| | automatización del entorno reproducible con Docker Compose. | y 34,5 h gestionadas. |
| Richard Rodríguez<br>Product Owner y Analista | Priorización del backlog del Sprint 2, definición de las 5 historias de usuario y validación del incremento contra los criterios de aceptación. | TeamTrack, OpenAPI. Métrica: 5 historias de usuario con criterios verificados. |
| Edgar Rosario<br>Dev Lead y Backend | Modelo de datos con 20 tablas y 4 migraciones, autenticación con hashing y token de sesión, construcción de los 29 endpoints de la API y catálogo de errores. | Java, Maven, JUnit, PostgreSQL, Flyway. Métrica: 24,5 h y 29 rutas de API publicadas. |
| Jonathan Gómez<br>Frontend y UX | Interfaz de aplicación única con 46 vistas, 53 rutas hash, formularios de autenticación y reconstrucción de los flujos de captura, consulta y seguimiento. | HTML, CSS y JavaScript. Métrica: 46 vistas (27 públicas y 19 internas) y 53 rutas. |
| Vivian Paris<br>QA Lead | Diseño de los casos de prueba, ejecución de la suite automatizada y validación de la matriz de autorización por rol contra la API en ejecución. | JUnit, Postman, Docker. Métrica: 68 pruebas en verde sobre 11 clases de prueba. |

### 4.5. Incremento Técnico Entregado

El Sprint 2 entrega el primer incremento ejecutable y persistente de la plataforma. Un usuario se autentica con credenciales reales, el servidor valida la sesión y aplica la matriz de autorización por rol antes de responder, y todas las operaciones de negocio quedan registradas sobre el esquema PostgreSQL versionado. El incremento es verificable de punta a punta con el arranque en contenedores.

- Autenticación con hashing BCrypt, token de sesión y respuestas 401 diferenciadas: NO_AUTENTICADO y CREDENCIALES_INVALIDAS.
- Autorización por rol verificada sobre las 29 rutas de la API, con el detalle publicado en la matriz de autorización del repositorio.
- Esquema de datos de 20 tablas con 4 migraciones Flyway y 9 usuarios semilla en 5 roles.
- Manejo de errores estandarizado con el catálogo de endpoints de error, incluida la respuesta 400 ENTRADA_INVALIDA.
- Entorno de ejecución en contenedores con health checks, arranque en un solo comando y variables de entorno documentadas.
- Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron Se reportaron 68 pruebas automatizadas sobre 11 clases, en verde, cubriendo autenticación, autorización, acceso a datos y servicios. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe

reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando. Esta métrica es histórica del corte del Sprint 2 y debe reproducirse con rama, fecha, commit y comando.

- Documentación técnica de API, autorización, errores y evidencia de cierre del Sprint en el repositorio.

### 4.6. Evidencias de Calidad, Seguridad y DevOps

| Área | Evidencia del Sprint 2 | Siguiente actividad |
|---|---|---|
| Calidad (testing) | 68 pruebas automatizadas en 11 clases, 0 fallos y 0 errores en la última ejecución con ./mvnw test. | Publicar el informe de cobertura y la calidad estática de SonarQube en el Sprint 3. |
| Seguridad | Matriz de autorización por rol verificada endpoint por endpoint, con códigos 401 y 403 reproducidos y catálogo de errores estandarizado. | Ejecutar OWASP ZAP sobre la interfaz y el backend, y registrar el reporte de hallazgos. |
| DevOps | Imágenes Docker de la API y la interfaz con health checks, arranque en un solo comando y variables de entorno documentadas. | Incorporar el pipeline de integración continua y las tareas de seguridad automatizadas. |
| Trazabilidad | 31 ítems cerrados en TeamTrack con responsable, esfuerzo y evidencia; 34 story points y 34,5 h de esfuerzo real grupal. | Cargar el Sprint 3 en TeamTrack y mantener la actualización diaria del registro. |

### 4.7. Métricas y Riesgos Actualizados

| Indicador | Valor o estado | Uso en el seguimiento |
|---|---|---|
| Historias de usuario cerradas | 5 de 5 (HU-10 a HU-14) | Confirma el alcance comprometido del Sprint 2. |
| Tareas cerradas | 26 de 26 | Mide el detalle operativo del Sprint. |
| Story points completados | 34 de 34 | Compara la velocidad con los 50 puntos del Sprint 1. |
| Horas de esfuerzo real grupal | 34 puntos x 2 h/punto = 68 h de valoración; TeamTrack registra 34,5 h de ejecución | Base de la facturación de consultoría del Sprint. |
| Velocidad del equipo | 34 SP en el Sprint 2, luego de 38 en el Sprint 1 | Revisar el tamaño de las historias antes del Sprint 3. |
| Cobertura de pruebas | 68 pruebas en 11 clases, 0 fallos y 0 errores | Línea base de calidad del Sprint 3. |
| Rutas con autorización verificada | 29 de 29 con matriz por rol publicada | Control de seguridad sobre la API. |
| Composición del código | Backend 69 clases y 5.032 líneas; interfaz 71 archivos y 13.959 líneas | Base para el cálculo de KLOC y de la velocidad de entrega. |
| Complejidad del modelo de datos | 20 tablas, 4 migraciones y 9 usuarios semilla en 5 roles | Control del riesgo de migraciones y del dominio. |

## Riesgos vigentes

| Riesgo | Señal de alerta | Acción |
|---|---|---|
| Autenticación y autorización concentradas en el backend sin análisis de calidad estática. | La calidad estática todavía no está publicada en el repositorio. | Ejecutar SonarQube sobre backend e interfaz en el Sprint 3 y corregir los hallazgos de mayor severidad. |
| Superficie expuesta sin análisis | La seguridad se valida hoy de | Incorporar OWASP ZAP al |

| | | |
|---|---|---|
| de seguridad automatizado. | forma manual, con casos de prueba. | pipeline y registrar el reporte de vulnerabilidades. |
| Comportamiento del sistema bajo concurrencia no evidenciado. | No existe plan de pruebas de carga sobre los endpoints de consulta. | Construir y ejecutar el plan de Apache JMeter con los datos reales del Sprint. |
| Caída de la velocidad del equipo. | El Sprint 2 bajó de 38 a 34 story points por el peso del modelado de datos. | Ajustar el compromiso del Sprint 3 a la capacidad observada en el Sprint 2. |

### 4.8. Retrospectiva del Sprint 2

El Sprint 2 consolidó el salto de prototipo a sistema verificable. El balance del equipo es que la decisión de trabajar contra el entorno completo en contenedores desde el primer día fue la que más aportó: cada historia se cerraba con una evidencia reproducible y no con una captura de pantalla, lo que además permitió detectar a tiempo el desbalance de la facturación.

- Lo que nos ayudó: el arranque en un solo comando con health checks, porque permitió repetir la verificación completa sin configuración manual.
- Lo que nos costó: el modelado de datos, porque separar la información de negocio de la información de auditoría condicionó el diseño de las 20 tablas y concentró el esfuerzo en el backend.
- Lo que mejoramos: automatizar la calidad estática y el análisis de seguridad, que hoy siguen dependiendo de verificación manual; SonarQube, OWASP ZAP y JMeter quedan comprometidos para el Sprint 3.
- Lo que mantuvimos: el registro diario en TeamTrack y la actualización de la evidencia en el repositorio, que resultaron más útiles de lo esperado.

Compromiso del Sprint 3: publicar las métricas de calidad estática y seguridad junto con el plan de pruebas de carga, de modo que el informe del Sprint no dependa únicamente de la verificación manual.

### 4.9. Factura de Consultoría del Sprint 2

Siguiendo la metodología establecida en el Sprint 1, la facturación del Sprint 2 se calcula sobre las horas de trabajo real grupal registradas en TeamTrack y se descuenta del presupuesto operativo de la empresa, aplicando la reserva del 30% definida en clase. El presupuesto operativo asciende a 350.000 Neuronix, una vez descontados los 450.000 Neuronix de la subasta y los 200.000 Neuronix reservados para herramientas.

| Concepto | Detalle |
|---|---|
| Horas de trabajo real grupal del Sprint 2 | 34 puntos x 2 h/punto = 68 h de valoración; TeamTrack registra 34,5 h de ejecución |
| Tarifa de la hora de consultoría | 412 Neuronix por hora |
| Factura del Sprint 2 | 34 puntos x 2 h/punto = 68 h x 412 Neuronix = 28.016 Neuronix |
| Reserva de la empresa (30%) | 8.405 Neuronix |
| Fondo operativo del equipo (70%) | 19.611 Neuronix |
| Saldo operativo antes del Sprint 2 | 318.688 Neuronix |
| Saldo operativo después del Sprint 2 | 290.672 Neuronix |
| Fondos de la subasta | 450.000 Neuronix, separados del presupuesto operativo |
| Fondo de herramientas | 200.000 Neuronix, reservado para el sostenimiento del equipo |
| Consumo del presupuesto operativo | 8,5% de los 350.000 Neuronix al cierre del Sprint 2 |
| Renglón de control | Control de presupuesto: 28.016 Neuronix facturados en el Sprint 2 |

Nota de redondeo: la presentación del Sprint 2 muestra los mismos conceptos redondeados para legibilidad (28.016, 8.405 y 19.611 Neuronix). El valor contable exacto de la factura es 28.016 Neuronix, resultado de 34 puntos x 2 horas x 412 Neuronix. Las 34,5 horas de TeamTrack representan ejecución registrada.

*Distribución económica sujeta a la aprobación del Sprint y a la regla de reserva del 30% definida en clase.*

## 5. Herramientas Recomendadas

### 5.1. Calidad (Testing)

- Selenium
- JUnit
- Postman
- Cypress
- Robot Framework
- Apache JMeter
- TestLink
- Katalon Studio
- Appium
- Qase

### 5.2. Seguridad

- OWASP ZAP
- Burp Suite Community
- Nmap
- Metasploit Framework
- OpenVAS
- Nikto
- Wfuzz
- SQLMap
- Aircrack-ng
- ClamAV

### 5.3. DevOps / CI/CD

- GitLab CI

- GitHub Actions
- Jenkins
- Docker
- Kubernetes
- Ansible
- Terraform
- SonarQube
- Prometheus
- Grafana

### 5.4. Aplicación de las herramientas en el Sprint 2

De la lista de herramientas recomendada para el proyecto, las siguientes se aplicaron efectivamente durante el Sprint 2. El estado refleja la situación al cierre del Sprint.

| Herramienta | Área | Aplicación en el Sprint 2 | Estado |
|---|---|---|---|
| JUnit | Calidad (testing) | Suite de 68 pruebas en 11 clases sobre autenticación, autorización, datos y servicios. | Completado |
| Postman | Calidad (testing) | Validación manual de los códigos 200, 401 y 403 de la matriz de autorización por rol. | Completado |
| OpenAPI | Calidad (testing) | Especificación de las 29 rutas y 37 operaciones del API, publicada en el repositorio. | Completado |
| Docker | DevOps / CI/CD | Base reproducible de | Completado |

| | | | |
|---|---|---|---|
| | | tres servicios con health checks, arranque en un solo comando y mapeo de puertos documentado. | |
| Git y GitHub | DevOps / CI/CD | Versionado del código, de la documentación técnica y de la evidencia de cierre del Sprint. | Completado |
| TeamTrack | Gestión | Registro de 31 ítems cerrados, 34 story points y 34,5 h de esfuerzo real grupal. | Completado |
| SonarQube | DevOps / CI/CD | Análisis de calidad estática del backend y de la interfaz: cobertura, deuda técnica y code smells. | En preparación |
| OWASP ZAP | Seguridad | Análisis de las superficies expuestas de la interfaz y del API, con reporte de hallazgos versionado. | En preparación |
| Apache JMeter | Calidad (testing) | Plan de pruebas de carga sobre los endpoints de consulta de uso frecuente. | En preparación |

## 6. Rúbrica de Evaluación

| Criterio | Puntos |
|---|---|
| Calidad del Software | 25 |

| | |
|---|---|
| Seguridad del Desarrollo | 25 |
| Agilidad y DevOps | 25 |
| Gestión del Proyecto | 25 |

## 7. Entrega Final

La entrega final debe incluir:

- Documento completo del proyecto
- Evidencias de todos los sprints
- Video de presentación (máx. 10 minutos)
- Lecciones aprendidas

## 8. Reflexión Final

**¿Cómo impacta este proyecto en la vida real de un consultor en ingeniería de software?**